

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.



LAB REPORT

on

Object Oriented Java Programming (23CS3PCOOJ)

Submitted by

Saraswati Yadav (**1BF24CS272**)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

In

B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Aug-2025 to Jan-2026

B.M.S. College of Engineering,

Bull Temple Road, Bangalore 560019

(Affiliated To Visvesvaraya Technological University, Belgaum)

Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled "Object Oriented Java Programming (23CS3PCOOJ)" carried out by **Saraswati Yadav (1BF24CS272)**, who is bonafide student of **B.M.S. College of Engineering**. It is in partial fulfilment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum. The Lab report has been approved as it satisfies the academic requirements in respect of an Object-Oriented Java Programming (23CS3PCOOJ) work prescribed for the said degree.

Dr. Seema Patil Associate Professor Department of CSE, BMSCE	Dr. Kavitha Sooda Professor & HOD Department of CSE, BMSCE
--	--

Index

Sl. No.	Date	Experiment Title	Page No.
1	23/09/2025	Develop a Java program that prints all real solutions to the quadratic equation $ax^2 + bx + c = 0$. Read in a, b, c and use the quadratic formula. If the discriminate $b^2 - 4ac$ is negative, display a message stating that there are no real solutions.	
2	14/10/2025	Develop a Java program to create a class Student with members usn, name, an array credits and an array mark. Include methods to accept and display details and a method to calculate SGPA of a student.	
3	14/10/2025	Create a class Book which contains four members: name, author, price, num_pages. Include a constructor to set the values for the members. Include methods to set and get the details of the objects. Include a toString() method that could display the complete details of the book. Develop a Java program to create n book objects.	
4	04/11/2025	Develop a Java program to create an abstract class named Shape that contains two integers and an empty method named printArea(). Provide three classes named Rectangle, Triangle and Circle such that each one of the classes extends the class Shape. Each one of the classes contain only the method printArea() that prints the area of the given shape.	
5	04/11/2025	Develop a Java program to create a class Bank that maintains two kinds of account for its customers, one called savings account and the other current account. The savings account provides compound interest and withdrawal facilities but no cheque book facility. The current account provides cheque book facility but no interest. Current account holders should also maintain a minimum balance and if the balance falls below this level, a service charge is imposed. Create a class Account that stores customer name, account number and type of account. From this derive the classes Cur-acct and Sav-acct to make them more specific to their requirements. Include the necessary methods in order to achieve the following tasks: a) Accept deposit from customer and update the balance. b) Display the balance. c) Compute and deposit interest d) Permit withdrawal and update the balance Check for the minimum balance, impose penalty if necessary and update the balance.	
6	18/11/2025	Create a package CIE which has two classes- Student and Internals. The class Personal has members like usn, name, sem. The class internals has an array	

		that stores the internal marks scored in five courses of the current semester of the student. Create another package SEE which has the class External which is a derived class of Student. This class has an array that stores the SEE marks scored in five courses of the current semester of the student. Import the two packages in a file that declares the final marks of n students in all five courses.	
7	25/11/2025	Write a program that demonstrates handling of exceptions in inheritance tree. Create a base class called "Father" and derived class called "Son" which extends the base class. In Father class, implement a constructor which takes the age and throws the exception WrongAge() when the input age<0. In Son class, implement a constructor that uses both father and son's age and throws an exception if son's age is >=father's age.	
8	09/12/2025	Write a program which creates two threads, one thread displaying "BMS College of Engineering" once every ten seconds and another displaying "CSE" once every two seconds.	
9	09/12/2025	Write a program that creates a user interface to perform integer divisions. The user enters two numbers in the text fields, Num1 and Num2. The division of Num1 and Num2 is displayed in the Result field when the Divide button is clicked. If Num1 or Num2 were not an integer, the program would throw a NumberFormatException. If Num2 were Zero, the program would throw an Arithmetic Exception Display the exception in a message dialog box.	
10	09/12/2025	Demonstrate Inter-process Communication and deadlock	

Program 1:

Develop a Java program that prints all real solutions to the quadratic equation $ax^2 + bx + c = 0$. Read in a, b, c and use the quadratic formula. If the discriminate $b^2 - 4ac$ is negative, display a message stating that there are no real solutions.

OBSERVATION:

Implement Quadratic Equation

Lab program 1 (Quadratic Equation)

```
import java.util.Scanner;
class Quadratic {
    public static void main (String[] args) {
        Scanner SC = new Scanner (System.in);
        System.out.println ("enter a");
        double a = SC.nextDouble();

        System.out.println ("enter b");
        double b = SC.nextDouble();

        System.out.println ("enter c");
        double c = SC.nextDouble();

        if (a == 0) {
            System.out.println ("Not a quadratic equation");
        } else {
            double d = b*b - 4*a*c;
            double r1, r2;
            if (d == 0) {
                r1 = (-b)/(2*a);
                System.out.println ("Roots are real and equal");
                System.out.println ("Root = " + r1);
            } else if (d > 0) {
                r1 = ((-b) + Math.sqrt(d))/(2*a);
                r2 = ((-b) - Math.sqrt(d))/(2*a);
                System.out.println ("Roots are real and distinct.");
                System.out.println ("Root 1 is " + r1 +
                                    "Root 2 is " + r2);
            } else {
                System.out.println ("Roots are imaginary");
            }
        }
    }
}
```


Code:

```
import java.util.Scanner;
```

```
class Quadratic {
```

```
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        System.out.println("enter a");
```

```
        double a = sc.nextDouble();
```

```
        System.out.println("enter b");
```

```
        double b = sc.nextDouble();
```

```
        System.out.println("enter c");
```

```
        double c = sc.nextDouble();
```

```
        if (a == 0) {
```

```
            System.out.println("Not a quadratic equation");
```

```
        } else {
```

```
            double d = b * b - 4 * a * c;
```

```
            double r1, r2;
```

```
            if (d == 0) {
```

```
                r1 = (-b) / (2 * a);
```



```

        System.out.println("Roots are real and equal.");
        System.out.println("Root = " + r1);
    } else if (d > 0) {
        r1 = ((-b) + Math.sqrt(d)) / (2 * a);
        r2 = ((-b) - Math.sqrt(d)) / (2 * a);
        System.out.println("Roots are real and distinct0.");
        System.out.println("Root 1 is " + r1 + " and Root 2 is " + r2);
    } else {
        System.out.println("Roots are imaginary");
        r1 = -b / (2 * a);
        r2 = Math.sqrt(-d) / (2 * a);
        System.out.println("Root 1 is " + r1 + " + " + r2 + "i");
        System.out.println("Root 2 is " + r1 + " - " + r2 + "i");
    }
}

sc.close();
}
}

```

Program 2:

Develop a Java program to create a class Student with members usn, name, an array credits and an array mark. Include methods to accept and display details and a method to calculate SGPA of a student.

OBSERVATION:

LAB - 2

Page No.:
Date:

Develop a Java program to create a class Student with members USN, name, an array credits an array marks. Include methods to accept and display details and a method to calculate SGPA of a student.

```
import java.util.Scanner;
```

```
class Subject {  
    int subjectMarks;  
    int credits;  
    int grade;  
}
```

```
class Student {  
    String name;  
    String USN;  
    double SGPA;  
    Scanner s;  
    Subject subject[];
```

```
    Student () {  
        s = new Scanner(System.in);  
        subject = new Subject[8];  
        for (int i = 0; i < 8; i++) {  
            subject[i] = new Subject();  
        }  
    }
```

```
    void getStudentDetails () {  
        System.out.print("Enter Student Name:");  
        name = s.nextLine();  
        System.out.print("Enter USN:");
```

```
usn = S.nextLine();
```

```
{
```

```
void getMarks() {
```

```
for (int i = 0; i < 8; i++) {
```

```
System.out.println("Subject " + (i+1) + ":");
```

```
while (true) {
```

```
System.out.print("Enter mark (0 to 100):");
```

```
subject[i].SubjectMarks = S.nextInt();
```

```
if (subject[i].SubjectMarks > 100) {
```

```
System.out.println("Marks cannot be more  
than 100. Please re-enter.");
```

```
} else if (subject[i].SubjectMarks < 0) {
```

```
System.out.println("Marks cannot be negative.  
Please re-enter.");
```

```
} else {
```

```
break;
```

```
}
```

```
}
```

```
System.out.print("Enter credits:");
```

```
subject[i].credits = S.nextInt();
```

```
subject[i].grade = (subject[i].SubjectMarks  
/ 10) + 1;
```

```
if (subject[i].grade == 11) {
```

```
subject[i].grade = 10;
```

```
}
```

```
if (subject[i].SubjectMark < 40) {
```

```
subject[i].grade = 0;
```

```
}
```

```
}
```

```
S.nextLine();
```

```
}
```

```
void compute SGPA() {
```

```
int effectiveScore = 0;
```


Page No.:
Date:

```

int totalCredits = 0;
for (int i = 0; i < 8; i++) {
    effectiveScore += (subject[i].grade * subject
        [i].credits);
    totalCredits += subject[i].credits;
}

```

```

SGPA = (double) effectiveScore / (double) totalCredits;
}

```

```

void display() {
    System.out.println("\n Student Details?");
    System.out.println("Name:" + name);
    System.out.println("USN:" + usn);
    System.out.printf("SGPA: %.2f\n", SGPA);
}
}

```

```

public class StudentSGPA {
    public static void main (String [] args) {
        Scanner scanner = new Scanner (System.in);
        System.out.print ("Enter number of students?");
        int n = scanner.nextInt();
        scanner.nextLine();
    }
}

```

```

Student [] students = new Student[n];
for (int i = 0; i < n; i++) {
    System.out.println("\n Enter details for student
        + (i+1) + ":");
    students[i] = new Student();
    students[i].getStudentDetails();
    students[i].getMarks();
    students[i].computeSGPA();
}
}

```

```

System.out.println("\n --- Student Results ---");
for (int i = 0; i < n; i++) {
}
}

```

```
students[i].display();  
}  
scanner.close();  
}
```

Output:

Enter num ber of Students: 2.

Enter details for student 1:

Enter Student Name: Alex

Enter USN: 1BF24CS272

Subject 1:

Enter marks (0 to 100): 97

Enter credits: 4

Subject 2:

Enter marks (0 to 100): 89

Enter credits: 3

Subject 3:

Enter marks (0 to 100): 95

Enter credits: 4

Subject 4:

Enter marks (0 to 100): 90

Enter credits: 3

Subject 5:

Enter marks (0 to 100): 88

Enter credits: 1

Subject 6:

Enter marks (0 to 100): 93

Enter credits: 3

Subject 7:

Enter marks (0 to 100): 87

Enter credits: 3

Code:

```
import java.util.Scanner;
```

```
class Subject {  
    int subjectMarks;  
    int credits;  
    int grade;  
}
```

```
class Student {  
    String name;  
    String usn;  
    double SGPA;  
    Scanner s;  
    Subject subject[];  
  
    Student() {  
        s = new Scanner(System.in);  
        subject = new Subject[8];  
        for (int i = 0; i < 8; i++) {  
            subject[i] = new Subject();  
        }  
    }  
}
```

```
void getStudentDetails() {
```

```
    System.out.print("Enter Student Name: ");
```

```
    name = s.nextLine();
```

```
    System.out.print("Enter USN: ");
```

```
    usn = s.nextLine();
```

```
}
```

```
void getMarks() {
```

```
    for (int i = 0; i < 8; i++) {
```

```
        System.out.println("Subject " + (i + 1) + ":");
```

```
        while (true) {
```

```
            System.out.print("Enter marks (0 to 100): ");
```

```
            subject[i].subjectMarks = s.nextInt();
```

```
            if (subject[i].subjectMarks > 100) {
```

```
                System.out.println("Marks cannot be more than 100. Please re-enter.");
```

```
            } else if (subject[i].subjectMarks < 0) {
```

```
                System.out.println("Marks cannot be negative. Please re-enter.");
```

```
            } else {
```

```
                break;
```

```
            }
```

```
        }
```

```
        System.out.print("Enter credits: ");
```

```
        subject[i].credits = s.nextInt();
```

```
        subject[i].grade = (subject[i].subjectMarks / 10) + 1;
```

```

        if (subject[i].grade == 11) {
            subject[i].grade = 10;
        }
        if (subject[i].subjectMarks < 40) {
            subject[i].grade = 0;
        }
    }
    s.nextLine();
}

void computeSGPA() {
    int effectiveScore = 0;
    int totalCredits = 0;

    for (int i = 0; i < 8; i++) {
        effectiveScore += (subject[i].grade * subject[i].credits);
        totalCredits += subject[i].credits;
    }

    SGPA = (double) effectiveScore / (double) totalCredits;
}

void display() {
    System.out.println("\nStudent Details:");
    System.out.println("Name: " + name);
    System.out.println("USN: " + usn);
    System.out.printf("SGPA: %.2f\n", SGPA);
}

```

```
}  
}
```

```
public class StudentSGPA {  
    public static void main(String[] args) {  
        Scanner scanner = new Scanner(System.in);  
        System.out.print("Enter number of students: ");  
        int n = scanner.nextInt();  
        scanner.nextLine();  
  
        Student[] students = new Student[n];  
  
        for (int i = 0; i < n; i++) {  
            System.out.println("\nEnter details for student " + (i + 1) + ":");  
            students[i] = new Student();  
            students[i].getStudentDetails();  
            students[i].getMarks();  
            students[i].computeSGPA();  
        }  
  
        System.out.println("\n--- Student Results ---");  
        for (int i = 0; i < n; i++) {  
            students[i].display();  
        }  
  
        scanner.close();  
    }  
}
```



```
    }  
}import java.util.Scanner;  
  
class Subject {  
    int subjectMarks;  
    int credits;  
    int grade;  
}  
  
class Student {  
    String name;  
    String usn;  
    double SGPA;  
    Scanner s;  
    Subject subject[];  
  
    Student() {  
        s = new Scanner(System.in);  
        subject = new Subject[8];  
        for (int i = 0; i < 8; i++) {  
            subject[i] = new Subject();  
        }  
    }  
}
```

```

void getStudentDetails() {
    System.out.print("Enter Student Name: ");
    name = s.nextLine();
    System.out.print("Enter USN: ");
    usn = s.nextLine();
}

```

```

void getMarks() {
    for (int i = 0; i < 8; i++) {
        System.out.println("Subject " + (i + 1) + ":");
        while (true) {
            System.out.print("Enter marks (0 to 100): ");
            subject[i].subjectMarks = s.nextInt();
            if (subject[i].subjectMarks > 100) {
                System.out.println("Marks cannot be more than 100. Please re-
enter.");
            } else if (subject[i].subjectMarks < 0) {
                System.out.println("Marks cannot be negative. Please re-enter.");
            } else {
                break;
            }
        }
        System.out.print("Enter credits: ");
        subject[i].credits = s.nextInt();

        subject[i].grade = (subject[i].subjectMarks / 10) + 1;
    }
}

```

```

        if (subject[i].grade == 11) {
            subject[i].grade = 10;
        }
        if (subject[i].subjectMarks < 40) {
            subject[i].grade = 0;
        }
    }
    s.nextLine();
}

void computeSGPA() {
    int effectiveScore = 0;
    int totalCredits = 0;

    for (int i = 0; i < 8; i++) {
        effectiveScore += (subject[i].grade * subject[i].credits);
        totalCredits += subject[i].credits;
    }

    SGPA = (double) effectiveScore / (double) totalCredits;
}

void display() {
    System.out.println("\nStudent Details:");
    System.out.println("Name: " + name);
    System.out.println("USN: " + usn);
    System.out.printf("SGPA: %.2f\n", SGPA);
}

```

```
}  
}
```

```
public class StudentSGPA {  
    public static void main(String[] args) {  
        Scanner scanner = new Scanner(System.in);  
        System.out.print("Enter number of students: ");  
        int n = scanner.nextInt();  
        scanner.nextLine();  
  
        Student[] students = new Student[n];  
  
        for (int i = 0; i < n; i++) {  
            System.out.println("\nEnter details for student " + (i + 1) + ":");  
            students[i] = new Student();  
            students[i].getStudentDetails();  
            students[i].getMarks();  
            students[i].computeSGPA();  
        }  
  
        System.out.println("\n--- Student Results ---");  
        for (int i = 0; i < n; i++) {  
            students[i].display();  
        }  
  
        scanner.close() } }
```

Program 3:

Create a class Book which contains four members: name, author, price, num_pages. Include a constructor to set the values for the members. Include methods to set and get the details of the objects. Include a toString() method that could display the complete details of the book. Develop a Java program to create n book objects

OBSERVATION:

LAB Program #3

Create a class Book which contains four members name, author, price, num-pages. Include a constructor to set the values for the members. Include methods to set and get the details of the objects. Include a toString() method that would display the complete details of the book. Develop a Java program to create n book objects.

```
import java.util.Scanner;
```

```
class Books {
```

```
    String name;
```

```
    String author;
```

```
    int price;
```

```
    int num-pages;
```

```
    Books (String name, String author, int price, int num-pages)
```

```
    {
```

```
        this.name = name;
```

```
        this.author = author;
```

```
        this.price = price;
```

```
        this.num-pages = num-pages;
```

```
    }
```

```
    public String toString()
```

```
    {
```

```
        String name = "Book name: " + this.name + "\n";
```

```
        String author = "Author name: " + this.author + "\n";
```

```
        String price = "price: " + this.price + "\n";
```

```
        String num-pages = "Number of pages: " + this.num-pages + "\n";
```

return name + author + price + num-pages ;

}

public class Main

{
public static void main (String args [])

{

Scanner in = new Scanner (System.in);

String name, author;

int price, num-pages;

System.out.print ("Enter number of books:");

int n = in.nextInt ();

Books [] b = new Books [n];

for (int i = 0; i < n; i++)

{

System.out.println ("In In Enter details
for book" + (i+1) + ":");

System.out.print ("Name:");

name = in.next ();

System.out.print ("Author:");

author = in.next ();

System.out.print ("price:");

price = in.nextInt ();

System.out.print ("Number of pages:");

num-pages = in.nextInt ();

b[i] = new Books (name, author, price, num-
pages);

}

System.out.println ("In In Book details: \n");

for (int i = 0; i < n; i++)


```

{
    System.out.println ( b[i].toString ());
}
in.close ();
}
}

```

Output:

Enter number of books: 2

Enter details for book 1:

Name : Harry-Potter

Author : JK - Rowling

Price : 450

Number of Pages : 223

Enter details for book 2:

Name : Percy - Jackson

Author : Rick - Riordan

Price : 399

Number of Pages : 350

Book details :

Book name : Harry-potter

Author name : JK - Rowling

Price : 450

Number of Pages : 223

Book name : Percy - Jackson

Author name : Rick - Riordan

Price : 399

Number of Pages : 350

Code:

```
import java.util.Scanner;
```

```
class Books
```

```
{
```

```
    String name;
```

```
    String author;
```

```
    int price;
```

```
    int num_pages;
```

```
    Books(String name, String author, int price, int num_pages)
```

```
{
```

```
        this.name = name;
```

```
        this.author = author;
```

```
        this.price = price;
```

```
        this.num_pages = num_pages;
```

```
}
```

```
    public String toString()
```

```
{
```

```
        return "Book name : " + name + "\n"
```

```
            + "Author name : " + author + "\n"
```

```
            + "Price : " + price + "\n"
```

```
            + "Number of pages : " + num_pages + "\n";
```

```
}
```

```
}
```

```
public class Main
{
    public static void main(String args[])
    {
        Scanner in = new Scanner(System.in);

        System.out.print("Enter number of books : ");
        int n = in.nextInt();
        in.nextLine();

        Books[] b = new Books[n];

        for(int i = 0; i < n; i++)
        {
            System.out.println("\nEnter details for book " + (i + 1));

            System.out.print("Name : ");
            String name = in.nextLine();

            System.out.print("Author : ");
            String author = in.nextLine();

            System.out.print("Price : ");
            int price = in.nextInt();
```

```
        System.out.print("Number of pages : ");
        int num_pages = in.nextInt();
        in.nextLine();

        b[i] = new Books(name, author, price, num_pages);
    }

    System.out.println("\nBook details:\n");

    for(int i = 0; i < n; i++)
    {
        System.out.println(b[i]);
    }

    in.close();
}
}
```

Program 4:

Develop a Java program to create an abstract class named Shape that contains two integers and an empty method named printArea(). Provide three classes named Rectangle, Triangle and Circle such that each one of the classes extends the class Shape. Each one of the classes contain only the method printArea() that prints the area of the given shape.

OBSERVATION:

Lab prog - 4

Develop a Java Program to create an abstract class named shape that contains two integers and an empty method named print Area(). Provide three classes named Rectangle, Triangle and circle such that each one of the classes extends the class shape. Each one of the classes contain only the method print Area() that prints the area of the given shape.

```
import Java.util.Scanner;
```

```
class Input Scanner {  
    Scanner sc = new Scanner (System.in);  
}
```

```
abstract class shape extends InputScanner {  
    int x, y;  
    abstract void printArea ();  
}
```

```
class Rectangle extends shape {  
    void input () {  
        System.out.println ("Enter the dimensions of  
        the rectangle (length and  
        breadth):");
```

```
        x = sc.nextInt ();
```

```
        y = sc.nextInt ();
```

```
    }  
    void printArea () {  
        System.out.println ("Area of Rectangle = " + (x*y));
```

```
    }  
}
```

```
class Triangle extends Shape {
```

Page No. _____
 Date _____

```

void input() {
    System.out.println("Enter the dimension of  

x sc.next the circle (radius):");
    r = sc.nextInt();
}

void printArea() {
    System.out.println("Area of Circle = " + (3.14 *
    }
    }
  
```

```

public class AreaDemo {
    public static void main (String [] args) {
        Rectangle r = new Rectangle();
        r.input();
        r.printArea();

        Triangle t = new Triangle();
        t.input();
        t.printArea();

        Circle c = new Circle();
        c.input();
        c.printArea();
    }
}
  
```

Output:

Enter the dimensions of the rectangle (length and Breadth)

6 7

Area of Rectangle = 42

Enter the dimensions of the triangle (base and Height):

9 8

Code:

```
import java.util.Scanner;
```

```
class InputScanner {
```

```
    Scanner sc = new Scanner(System.in);
```

```
}
```

```
abstract class Shape extends InputScanner {
```

```
    int x, y;
```

```
    abstract void printArea();
```

```
}
```

```
class Rectangle extends Shape {
```

```
    void input() {
```

```
        System.out.println("Enter the dimensions of the rectangle (length and  
breadth):");
```

```
        x = sc.nextInt();
```

```
        y = sc.nextInt();
```

```
    }
```

```
    void printArea() {
```

```
        System.out.println("Area of Rectangle = " + (x * y));
```

```
    }
```

```
}
```

```
class Triangle extends Shape {
```

```
    void input() {
```

```
        System.out.println("Enter the dimensions of the triangle (base and height:");
```

```
        x = sc.nextInt();
```

```
        y = sc.nextInt();
```

```
    }
```

```
    void printArea() {
```

```
        System.out.println("Area of Triangle = " + (0.5 * x * y));
```

```
    }
```

```
}
```

```
class Circle extends Shape {
```

```
    void input() {
```

```
        System.out.println("Enter the dimension of the circle (radius:");
```

```
        x = sc.nextInt();
```

```
    }
```

```
    void printArea() {
```

```
        System.out.println("Area of Circle = " + (3.14 * x * x));
```

```
    }
```

```
}
```

```
public class AreaDemo {
```

```
    public static void main(String[] args) {
```

```
        Rectangle r = new Rectangle();
```

```
        r.input();
```

```
        r.printArea();
```

```
Triangle t = new Triangle();
```

```
t.input();
```

```
t.printArea();
```

```
Circle c = new Circle();
```

```
c.input();
```

```
c.printArea();
```

```
}
```

```
}
```

Program 5:

Develop a Java program to create a class Bank that maintains two kinds of account for its customers, one called savings account and the other current account. The savings account provides compound interest and withdrawal facilities but no cheque book facility. The current account provides cheque book facility but no interest. Current account holders should also maintain a minimum balance and if the balance falls below this level, a service charge is imposed. Create a class Account that stores customer name, account number and type of account. From this derive the classes Cur-acct and Sav-acct to make them more specific to their requirements. Include the necessary methods in order to achieve the following tasks:

- a) Accept deposit from customer and update the balance.
- b) Display the balance.
- c) Compute and deposit interest
- d) Permit withdrawal and update the balance Check for the minimum balance, impose penalty if necessary and update the balance.

OBSERVATION:

Lab Program : 5

Create a class Account that stores name, account number and type of account. From this derive the classes Cur-acc and Sav-acc to make them more specific to their requirements. Include

- Accept deposit from customer and update the balance.
- Display the balance.
- Compute and deposit interest.
- Permit withdrawal and update the balance.
- Check the minimum balance, impose penalty, if necessary.

```
import java.util.Scanner;
```

```
class Account {  
    String name, type;  
    int accNo;  
    double balance;  
    Scanner sc = new  
        Scanner (System.in);
```

```
    Account (String n, int a, String t) {  
        name = n;  
        accNo = a;  
        type = t;  
        balance = 0;  
    }
```

```
    void deposit () {  
        System.out.println ("Enter the deposit amount:");  
        double amt = sc.nextDouble();  
        balance += amt;  
    }
```

System.out.println ("Deposited : " + amt + "
Updated balance : " + balance);

}
void withdraw () {
 System.out.print ("Enter the withdrawl
 amount : ");
 double amt = sc.nextDouble();
 if (amt <= balance) {
 balance -= amt;
 }

System.out.println ("Withdrawn : " + amt +
Updated balance : " + balance);

} else {
 System.out.println ("Insufficient
 Balance!");
}

}

void display () {

 System.out.println ("Customer name : " +
 name);

 System.out.println ("Account number : " + accNo);

 System.out.println ("Type of Account : " + type);

 System.out.println ("Account Balance : " + balance);

}

}

class SavAcct extends Account {

 SavAcct (String n, String a) {

 super (n, a, "Saving");

 }

 void interest () {

 double i = balance * 0.04;

 balance += i;

Page No. _____
Date: _____

```
System.out.println("Interest added: "+i+"?  
Updated balance: "+balance);
```

```
}  
}
```

```
class CurrentAcct extends Account {  
    curAcct (String n, String a) {  
        Super (n, a, "current");  
    }  
}
```

```
void checkMin () {  
    if (balance < 1000) {  
        balance -= 50;  
        System.out.println ("Penalty Charged!  
Updated balance: "+bal  
ance);  
    }  
}
```

```
}  
}  
}
```

```
Public class BankDemo {  
    public static void main (String [] args)  
    {  
        Scanner sc = new Scanner (System.in);
```

```
System.out.print ("Enter customer name: ");  
String n1 = sc.nextLine();  
System.out.print ("Enter account number: ");  
String a1 = sc.nextLine();  
SavAcct s = new SavAcct (n1, a1);
```

```
System.out.print ("Enter customer name: ");  
String n2 = sc.nextLine();
```

```
System.out.print ("Enter account number: ");
```

Page No. _____
Date: _____

```
string a2 = sc.nextLine();  
curAcct c = new new CurAcct(n2, a2);
```

```
while (true) {  
    System.out.println("\n--Menu--");  
    System.out.println("1. Deposit 2. Withdraw  
3. Interest 4. Display  
5. Exit");  
    System.out.println("Choice:");  
    int ch = sc.nextInt();  
    sc.nextLine();
```

```
    System.out.print("Account type (savings/current);  
    String t = sc.nextLine().trim();
```

```
    Account acc = t.equalsIgnoreCase("Savings")?  
        S:C;
```

```
    switch (ch) {
```

```
        case 1:
```

```
            acc.deposit();
```

```
            break;
```

```
        case 2:
```

```
            acc.withdraw();
```

```
            break; if (acc instanceof CurAcct)
```

```
                ((CurAcct) acc).checkMin();
```

```
            break;
```

```
        case 3:
```

```
            if (acc instanceof SavAcct)
```

```
                ((SavAcct) acc).interest();
```

```
            else
```

```
                System.out.println("No interest for  
                current account.");
```

```
            break;
```


case 4:

acc.display();
break;

case 5:

System.out.println("Exiting program");
sc.close();

return;

default:

System.out.println("Invalid choice");

}

}

}

}

Output:

Enter customer name : Gloria

Enter account number : 801

Enter customer name : Claire

Enter account number : 802

--- MENU ---

1. Deposit 2. Withdraw 3. Interest 4. Display
5. Exit

choice : 1

Account type (saving/current): saving

Enter the deposit amount : 10,000

Deposited : 10000.0 Updated balance : 10000.0

--- MENU ---

1. Deposit 2. Withdraw 3. Interest 4. Display
5. Exit

choice : 1

Account type (saving/current): current

Code:

```
import java.util.Scanner;
```

```
class Account {
```

```
    String name, type, accNo;
```

```
    double balance;
```

```
    Scanner sc = new Scanner(System.in);
```

```
    Account(String n, String a, String t) {
```

```
        name = n;
```

```
        accNo = a;
```

```
        type = t;
```

```
        balance = 0;
```

```
    }
```

```
    void deposit() {
```

```
        System.out.print("Enter the deposit amount: ");
```

```
        double amt = sc.nextDouble();
```

```
        balance += amt;
```

```
        System.out.println("Deposited: " + amt + ". Updated balance: " + balance);
```

```
    }
```

```
    void withdraw() {
```

```
        System.out.print("Enter the withdrawal amount: ");
```

```
        double amt = sc.nextDouble();
```

```
        if (amt <= balance) {
```



```
        balance -= amt;

        System.out.println("Withdrawn: " + amt + ". Updated balance: " +
balance);
    } else {
        System.out.println("Insufficient Balance!");
    }
}
```

```
void display() {
    System.out.println("Customer name: " + name);
    System.out.println("Account number: " + accNo);
    System.out.println("Type of Account: " + type);
    System.out.println("Account Balance: " + balance);
}
}
```

```
class SavAcct extends Account {
    SavAcct(String n, String a) {
        super(n, a, "saving");
    }
}
```

```
void interest() {
    double i = balance * 0.04;
    balance += i;
    System.out.println("Interest added: " + i + ". Updated balance: " +
balance);
}
```

```
}
```

```
class CurAcct extends Account {
```

```
    CurAcct(String n, String a) {
```

```
        super(n, a, "current");
```

```
    }
```

```
    void checkMin() {
```

```
        if (balance < 1000) {
```

```
            balance -= 50;
```

```
            System.out.println("Penalty charged! Updated balance: " + balance);
```

```
        }
```

```
    }
```

```
}
```

```
public class BankDemo {
```

```
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        System.out.print("Enter customer name: ");
```

```
        String n1 = sc.nextLine();
```

```
        System.out.print("Enter account number: ");
```

```
        String a1 = sc.nextLine();
```

```
        SavAcct s = new SavAcct(n1, a1);
```

```
        System.out.print("Enter customer name: ");
```

```

String n2 = sc.nextLine();

System.out.print("Enter account number: ");

String a2 = sc.nextLine();

CurAcct c = new CurAcct(n2, a2);


while (true) {

    System.out.println("\n-----MENU-----");

    System.out.println("1.Deposit 2.Withdraw 3.Interest 4.Display
5.Exit");

    System.out.print("Choice: ");

    int ch = sc.nextInt();

    sc.nextLine(); // consume newline


    System.out.print("Account type (saving/current): ");

    String t = sc.nextLine().trim();

    Account acc = t.equalsIgnoreCase("saving") ? s : c;


    switch (ch) {

        case 1:

            acc.deposit();

            break;

        case 2:

            acc.withdraw();

            if (acc instanceof CurAcct)

                ((CurAcct) acc).checkMin();

            break;

```

```
case 3:
    if (acc instanceof SavAcct)
        ((SavAcct) acc).interest();
    else
        System.out.println("No interest for current account.");
    break;
case 4:
    acc.display();
    break;
case 5:
    System.out.println("Exiting program...");
    sc.close();
    return;
default:
    System.out.println("Invalid choice");
}
}
}
}
```

Program 6:

Create a package CIE which has two classes- Student and Internals. The class Personal has members like usn, name ,sem. The class internals has an array that stores the internal marks scored in five courses of the current semester of the student. Create another package SEE which has the class External which is a derived class of Student. This class has an array that stores the SEE marks scored in five courses of the current semester of the student. Import the two packages in a file that declares the final marks of n students in all five courses.

OBSERVATION:

Lab Prog-6

Create a package CIE which has two classes - students and Internals. The class students has members like USN, name, sem. The class internals derived from Student has an array that stores the internal marks scored in five courses of the current semester of the student. Create another package SEE which has the class External which is a derived class of student. This class has an array that stores the SEE marks scored in five courses of the current semester of the student. Import the two packages in a file that declares the final marks of n students in all five courses.

code:

```
Package CIE;
import java.util.Scanner;
public class Student {
    protected String USN;
    protected String name;
    protected int sem;

    public void inputStudentDetails() {
        Scanner S = new Scanner(System.in);

        System.out.print("Enter USN: ");
        USN = S.nextLine();

        System.out.print("Enter Name: ");
        name = S.nextLine();

        System.out.print("Enter Semester: ");
        sem = S.nextInt();
    }
}
```

2

```
public void display StudentDetails() {
    System.out.println ("USN: " + USN);
    System.out.println ("Name: " + name);
    System.out.println ("Semester: " + sem);
}
```

3

```
package CIE;
```

```
import java.util.Scanner;
```

```
public class Internals extends Student {
    protected void inputCIEmarks() {
        Scanner s = new Scanner(System.in);
        System.out.println ("Enter CIE
        marks for 5 subjects:");
```

```
for (int i = 0; i < 5; i++) {
    System.out.print ("CIE Mark for Subject " + (i+1)
    + ": ");
    cieMarks[i] = s.nextInt();
}
```

4

```
Package SEE;
```

```
import CIE.Internals;
```

```
import Java.util.Scanner;
```

```
public class Externals extends Internals {
    protected int seeMarks[];
    protected int finalMarks[];
```



```
public External() {
    SeeMarks = new int [5];
    final Marks = new int [5];
}
```

```
public void input SEEmarks () {
    Scanner S = new Scanner (System.in);
    System.out.println ("Enter SEE Marks
    for 5 subjects :");
```

```
for (int i = 0; i < 5; i++) {
    System.out.println ("SEE Mark for Subject"
    + (i+1) + " : ");
    SEEMarks [i] = s.nextInt();
}
```

```
public void calculate Final Marks () {
    for (int i = 0; i < 5; i++) {
        finalMarks [i] = cleMarks [i] + (See Marks [i]);
    }
}
```

```
public void display Final Marks () {
    System.out.println ("Final Marks
    display Student Details (); Report ---");
```

```
for (int i = 0; i < 5; i++) {
    System.out.println ("Final Mark (Subject
    " + (i+1) + " :
    final Marks
```

```
}
```

```
System.out.println ();
```

```
}
```

import SEE.~~int~~ Externals;
import Java.util. Scanner;

class Main {
 public static void main(String args[]) {
 Scanner S = new Scanner (System.in);

System.out.print("Enter number of students");
int n = S.nextInt();

Externals students [] = new Externals [n];

for (int i = 0; i < n; i++) {
 System.out.println("\n --- Enter details for
 Student " + (i+1) + " ---");
 students [i] = new Externals ();

students [i].inputStudentDetails();
students [i].input CIE marks ();
students [i].input SEE marks ();
students [i].calculate FinalMarks ();

}
System.out.println("\n ----- Final Marks OF
 ALL STUDENTS ---");

for (int i = 0; i < n; i++) {
 students [i].display FinalMarks ();

}
}
}

Output: Enter number of students: 1
---Enter details for Student 1---

Enter USN: S1
Enter Name: Phil
Enter Semester: 3

Enter CIE Marks for 5 subjects:
CIE Mark for subject 1: 50
CIE Mark for subject 2: 65
CIE Mark for subject 3: 45
CIE Mark for subject 4: 58
CIE Mark for subject 5: 56

Enter SEE Marks for 5 subjects:
SEE Mark for subject 1: 67
SEE Mark for subject 2: 78
SEE Mark for subject 3: 45
SEE Mark for subject 4: 87
SEE Mark for subject 5: 57

----- Final MARKS OF ALL STUDENTS -----

---Final Marks Report ---

USN: S1

Name: Phil

Semester: 3

Final Marks (Subject 1): 83

Final Marks (Subject 2): 104

Final Marks (Subject 3): 67

Final Marks (Subject 4): 101

Final Marks (Subject 5): 84

Code:

Internals:

```
package CIE;
```

```
import java.util.Scanner;
```

```
public class Internals extends Student {
```

```
    protected int cieMarks[] = new int[5];
```

```
    public void inputCIEmarks() {
```

```
        Scanner s = new Scanner(System.in);
```

```
        System.out.println("\nEnter CIE Marks for 5 subjects:");
```

```
        for (int i = 0; i < 5; i++) {
```

```
            System.out.print("CIE Mark for subject " + (i + 1) + ": ");
```

```
            cieMarks[i] = s.nextInt();
```

```
        }
```

```
    }
```

```
}
```

Students:

```
package CIE;
```

```
import java.util.Scanner;
```

```
public class Student {
```



```
protected String usn;  
protected String name;  
protected int sem;
```

```
public void inputStudentDetails() {  
    Scanner s = new Scanner(System.in);  
  
    System.out.print("Enter USN: ");  
    usn = s.nextLine();  
  
    System.out.print("Enter Name: ");  
    name = s.nextLine();  
  
    System.out.print("Enter Semester: ");  
    sem = s.nextInt();  
}
```

```
public void displayStudentDetails() {  
    System.out.println("USN: " + usn);  
    System.out.println("Name: " + name);  
    System.out.println("Semester: " + sem);  
}  
}
```

Externals:

package SEE;

import CIE.Internals;

import java.util.Scanner;

public class Externals extends Internals {

protected int seeMarks[];

protected int finalMarks[];

public Externals() {

seeMarks = new int[5];

finalMarks = new int[5];

}

public void inputSEEmarks() {

Scanner s = new Scanner(System.in);

System.out.println("\nEnter SEE Marks for 5 subjects:");

for (int i = 0; i < 5; i++) {

System.out.print("SEE Mark for subject " + (i + 1) + ": ");

seeMarks[i] = s.nextInt();

}

}

public void calculateFinalMarks() {

```

    for (int i = 0; i < 5; i++) {
        // CIE out of 50, SEE out of 100, so SEE/2
        finalMarks[i] = cieMarks[i] + (seeMarks[i] / 2);
    }
}

```

```

public void displayFinalMarks() {
    System.out.println("\n----- Final Marks Report -----");
    displayStudentDetails();
}

```

```

    for (int i = 0; i < 5; i++) {
        System.out.println("Final Marks (Subject " + (i + 1) + "): " +
finalMarks[i]);
    }
    System.out.println();
}
}

```

Main:

```

import SEE.Externals;
import java.util.Scanner;

```

```

class Main {

```

```

    public static void main(String args[]) {

```

```

        Scanner s = new Scanner(System.in);

```

```

System.out.print("Enter number of students: ");
int n = s.nextInt();

Externals students[] = new Externals[n];

for (int i = 0; i < n; i++) {
    System.out.println("\n--- Enter details for Student " + (i + 1) + " ---");
    students[i] = new Externals();

    students[i].inputStudentDetails();
    students[i].inputCIEMarks();
    students[i].inputSEEMarks();
    students[i].calculateFinalMarks();
}

System.out.println("\n===== FINAL MARKS OF ALL STUDENTS
=====\\n");

for (int i = 0; i < n; i++) {
    students[i].displayFinalMarks();
}
}
}

```

Program 7:

Write a program that demonstrates handling of exceptions in inheritance tree. Create a base class called "Father" and derived class called "Son" which extends the base class. In Father class, implement a constructor which takes the age and throws the exception WrongAge() when the input age<0. In Son class, implement a constructor that uses both father and son's age and throws an exception if son's age is >=father's age.

OBSERVATION:

LAB prog - 7

Write a program that demonstrates handling of Exceptions in inheritance tree. Create a base class called "Father" and derived class called "Son" which extends the base class. In Father class, implement a constructor which takes the age and throws the exception WrongAge() when the input age < 0 . In Son class, implement a constructor that takes both Father and son's age and throws an exception if son's age is $>$ Father's age.

code:

```
import java.util.Scanner;
class WrongAge extends Exception
{
    public WrongAge (String message)
    {
        super (message);
    }
}

class Father
{
    int FatherAge;
    Father (int age) throws WrongAge
    {
        if (age < 0)
        {
            throw new WrongAge ("Father's age cannot be negative");
        }
        this.FatherAge = age;
    }
}
```

```

3
void display ()
{
    System.out.println ("father's age : " + FatherAge);
}

3
class Son extends Father {
    int sonAge;

    Son (int FatherAge, int sonAge) throws WrongAge
    {
        super (FatherAge);

        if (sonAge >= FatherAge)
        {
            throw new WrongAge
            (
                "Son's age cannot be greater
                than or equal to Father's
                age !"
            );
        }

        if (sonAge < 0)
        {
            throw new WrongAge ("Son's age cannot
            be negative !");
        }

        this.sonAge = sonAge;
    }
}

3
public class Main
{
    public static void main (String [] args)

```



```

{
    Scanner in = new Scanner (System.in);
    int s_age ; f_age;
    System.out.println ("Enter Father's age");
    f_age = in.nextInt();
    System.out.println ("Enter Son's age");
    s_age = in.nextInt();
    try {
        Son s = new Son (f_age, s_age);
        System.out.println ("Father and Son
                                objects created
                                successfully!");
        s.display();
    }
    catch (WrongAge e)
    {
        System.out.println ("Wrong Age Error: "
                            + e.getMessage());
    }
}

```

Output:

```

Enter Father's age:
45
Enter Son's age:
12
Father and son objects created successfully!
Father's age: 45

Enter Father's age:
45
Enter Son's age:

```

For 20/12/21

Wite 2mm

Code:

```
import java.util.Scanner;
```

```
class WrongAge extends Exception
```

```
{  
    public WrongAge(String message)  
    {  
        super(message);  
    }  
}
```

```
class Father
```

```
{  
    int fatherAge;
```

```
    Father(int age) throws WrongAge
```

```
{  
    if (age<0)  
    {  
        throw new WrongAge("Father's age cannot be negative!");  
    }  
    this.fatherAge = age;  
}
```

```
void display()
```

```
{  
    System.out.println("Father's age : "+fatherAge);  
}
```

```
}  
}
```

```
class Son extends Father{  
    int sonAge;
```

```
    Son(int fatherAge,int sonAge) throws WrongAge
```

```
{  
    super(fatherAge);
```

```
    if (sonAge>=fatherAge)
```

```
    {  
        throw new WrongAge
```

```
        (  
            "Son's age cannot be greater than or equal to father's age!"  
        );
```

```
    }
```

```
    if (sonAge<0)
```

```
    {  
        throw new WrongAge("Son's age cannot be negative!");
```

```
    }
```

```
    this.sonAge=sonAge;
```

```
}
```



```
}
```

```
public class Main
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        Scanner in = new Scanner(System.in);
```

```
        int s_age,f_age;
```

```
        System.out.println("Enter father's age : ");
```

```
        f_age = in.nextInt();
```

```
        System.out.println("Enter son's age : ");
```

```
        s_age = in.nextInt();
```

```
        try{
```

```
            Son s = new Son(f_age,s_age);
```

```
            System.out.println("Father and Son objects created successfully!");
```

```
            s.display();
```

```
        }
```

```
        catch(WrongAge e)
```

```
        {
```

```
            System.out.println("Wrong Age Error: " + e.getMessage());
```

```
        }
```

```
    }
```

```
}
```

Program 8:

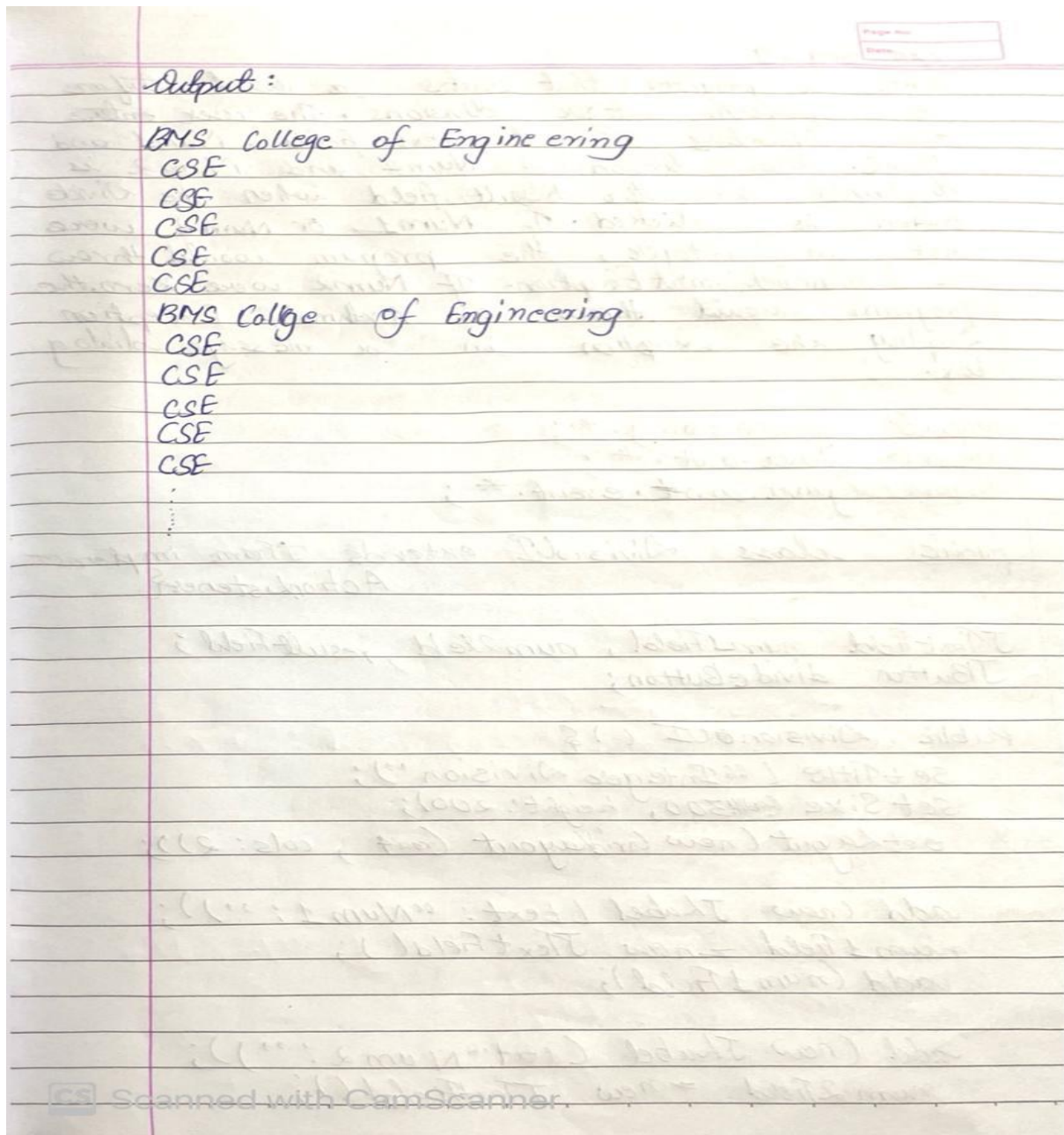
Write a program which creates two threads, one thread displaying “BMS College of Engineering” once every ten seconds and another displaying “CSE” once every two seconds.

OBSERVATION:

Lab Program no 8

Write a program which creates two threads one thread displaying "BMS college of Engineering" once every ten seconds and another displays "CSE" once every two seconds.

```
Public class ThreadDemo {  
    public static void main (String [] args) {  
        Thread t1 = new Thread () -> {  
            try {  
                while (true) {  
                    System.out.println ("BMS college  
Engineering");  
                    Thread.sleep (1000);  
                }  
            } catch (InterruptedException e) {  
                e.printStackTrace ();  
            }  
        };  
  
        Thread t2 = new Thread () -> {  
            try {  
                while (true) {  
                    System.out.println ("CSE");  
                    Thread.sleep (2000);  
                }  
            } catch (InterruptedException e) {  
                e.printStackTrace ();  
            }  
        };  
  
        t1.start ();  
        t2.start ();  
    }  
}
```



CODE:

```
public class ThreadDemo {  
  
    public static void main(String[] args) {  
  
        Thread t1 = new Thread(() -> {  
            try {  
                while (true) {
```

```
        System.out.println("BMS College of Engineering");
        Thread.sleep(10000);
    }
} catch (InterruptedException e) {
    e.printStackTrace();
}
});
```

```
Thread t2 = new Thread(() -> {
    try {
        while (true) {
            System.out.println("CSE");
            Thread.sleep(2000);
        }
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
});
```

```
t1.start();
t2.start();
}
}
```


Program 9:

Write a program that creates a user interface to perform integer divisions. The user enters two numbers in the text fields, Num1 and Num2. The division of Num1 and Num2 is displayed in the Result field when the Divide button is clicked. If Num1 or Num2 were not an integer, the program would throw a `NumberFormatException`. If Num2 were Zero, the program would throw an `ArithmeticException`. Display the exception in a message dialog box.

OBSERVATION:

LAB prog-9

Write a program that creates a user interface to perform integer divisions. The user enters two Numbers in the text fields, Num1 and Num2. The division of Num1 and Num2 is displayed in the Result field when the Divide button is clicked. If Num1 or Num2 were not an integer, the program would throw a NumberFormatException. If Num2 were Zero, the program would throw an ArithmeticException. Display the exception in a message dialog box.

```
import javax.swing.*.*;
import java.awt.*.*;
import java.awt.event.*;
```

```
public class DivisionUI extends JFrame implements
    ActionListener {
```

```
    JTextField num1Field, num2Field, resultField;
    JButton divideButton;
```

```
    public DivisionUI () {
```

```
        setTitle ("Integer Division");
```

```
        setSize (width: 300, height: 200);
```

```
        setLayout (new GridLayout (row: 4, cols: 2));
```

```
        add (new JLabel (text: "Num1:"));
```

```
        num1Field = new JTextField (1);
```

```
        add (num1Field);
```

```
        add (new JLabel (text: "Num2:"));
```

```
        num2Field = new JTextField (1);
```

add (num2Field);

add (new JLabel ("Result:"));
resultField = new JTextField();
resultField.setEditable (false);
add (resultField);

divide Button = new JButton ("Divide");
divide Button.addActionListener (this);
add (divide Button);

setDefaultCloseOperation (JFrame.EXIT_ON_CLOSE);
setVisible (true);
}

```
public void actionPerformed (ActionEvent e) {  
    try {  
        int num1 = Integer.parseInt (num1Field.  
                                            getText());  
        int num2 = Integer.parseInt (num2Field.  
                                            getText());  
        int result = num1 / num2;  
        resultField.setText (String.valueOf (result));  
    }  
    catch (NumberFormatException nfe) {  
        JOptionPane.showMessageDialog (this,  
            message: "Please enter valid integers!",  
            title: "Number Format Error",  
            JOptionPane.ERROR_MESSAGE);  
    }  
    catch (ArithmeticException ae) {  
        JOptionPane.showMessageDialog (this,  
            "Division by zero is not allowed",  
            "Arithmetic Error",
```


JOptionPane.ERROR_MESSAGE);

}
}

Public static void main(String[] args)
new DivisionDI();

}
}

Output:

Integer Division

Num 1: 16

Num 2: 4

Result: 4

Divide

CODE:

```
import javax.swing.*;
```

```
import java.awt.*;
```

```
import java.awt.event.*;
```

```
public class DivisionUI extends JFrame implements ActionListener {
```

```
    JTextField num1Field, num2Field, resultField;
```

```
    JButton divideButton;
```

```
    public DivisionUI() {
```

```
        setTitle("Integer Division");
```

```
        setSize(300, 200);
```

```
        setLayout(new GridLayout(4, 2));
```

```
        add(new JLabel("Num1:"));
```

```
        num1Field = new JTextField();
```

```
        add(num1Field);
```

```
        add(new JLabel("Num2:"));
```

```
        num2Field = new JTextField();
```

```
        add(num2Field);
```

```
        add(new JLabel("Result:"));
```

```
        resultField = new JTextField();
```

```
        resultField.setEditable(false);
```

```
        add(resultField);
```



```
divideButton = new JButton("Divide");
divideButton.addActionListener(this);
add(divideButton);

setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
setVisible(true);
}

public void actionPerformed(ActionEvent e) {
    try {
        int num1 = Integer.parseInt(num1Field.getText());
        int num2 = Integer.parseInt(num2Field.getText());

        int result = num1 / num2;
        resultField.setText(String.valueOf(result));

    } catch (NumberFormatException nfe) {
        JOptionPane.showMessageDialog(this,
            "Please enter valid integers!",
            "Number Format Error",
            JOptionPane.ERROR_MESSAGE);
    } catch (ArithmeticException ae) {
        JOptionPane.showMessageDialog(this,
            "Division by zero is not allowed!",
            "Arithmetic Error",
```

```
        JOptionPane.ERROR_MESSAGE);  
    }  
}  
  
public static void main(String[] args) {  
    new DivisionUI();  
}  
}
```

Program 10:

Demonstrate Inter process Communication and deadlock.

OBSERVATION:

Lab pgm 10: Open Ended Exercise

Demonstrate Interprocess Communication and deadlock

```
class Q {
    int n;
    boolean valueSet = false;
    synchronized int get() {
        while (!valueSet) {
            try {
                System.out.println("\n Consumer Waiting\n");
                wait();
            } catch (InterruptedException e) {
                System.out.println("InterruptedException
                                   caught");
            }
        }
        System.out.println("Got: " + n);
        valueSet = false;
    }

    System.out.println("\n Intimate Success Proceed\n");
    notify();
    return n;
}

Synchronized void put (int n) {
    while (valueSet) {
        try {
            System.out.println("\n Producer waiting\n");
            wait();
        } catch (InterruptedException e) {
            System.out.println("InterruptedException
                               caught");
        }
    }
}

class Producer implements Runnable {
```

Q q;
 Producer (Q q) {
 this.q = q;
 new Thread (this, "Producer").start();
}

{
 public void run () {
 int i = 0;
 while (i < 15) {
 q.put (i++);
 }
}

}
 class Consumer implements Runnable {
 Q q;
 Consumer (Q q) {
 this.q = q;
 new Thread (this, "Consumer").start();
 }

{
 public void run () {
 int i = 0;
 while (i < 15) {
 int r = q.get ();
 System.out.println ("Consumed : " + r);
 i++;
 }
}

}
 class PCFixed {
 public static void main (String args []) {
 Q q = new Q (1);
 new Producer (q);
 new Consumer (q);
 System.out.println ("Press Control -C to stop.");
 }

2
3

Output :

Press Control-C to stop.

Put : 0

Intimate Consumer

Producers waiting

Got : 0

Intimate Producer

Put : 1

Intimate Consumer

Producers waiting

consumed : 0

Got : 1

Intimate Producer

Put : 2

Intimate Consumer

Producers waiting

CS Consumed : 1
Got : 2.

CODE:

```
class Q {  
    int n;  
    boolean valueSet = false;  
    synchronized int get() {  
        while (!valueSet) {  
            try {  
                System.out.println("\nConsumer waiting\n");  
                wait();  
            } catch (InterruptedException e) {  
                System.out.println("InterruptedException caught");  
            }  
        }  
        System.out.println("Got: " + n);  
        valueSet = false;  
  
        System.out.println("\nIntimate Producer\n");  
        notify();  
        return n;  
    }  
    synchronized void put(int n) {  
        while (valueSet) {  
            try {  
                System.out.println("\nProducer waiting\n");  
                wait();  
            } catch (InterruptedException e) {
```

```

        System.out.println("InterruptedException caught");
    }
}

this.n = n;
valueSet = true;
System.out.println("Put: " + n);
System.out.println("\nIntimate Consumer\n");
notify();
}
}

class Producer implements Runnable {
    Q q;

    Producer(Q q) {
        this.q = q;
        new Thread(this, "Producer").start();
    }

    public void run() {
        int i = 0;
        while (i < 15) {
            q.put(i++);
        }
    }
}

class Consumer implements Runnable {
    Q q;

    Consumer(Q q) {

```

```

        this.q = q;
        new Thread(this, "Consumer").start();
    }
    public void run() {
        int i = 0;
        while (i < 15) {
            int r = q.get();
            System.out.println("consumed: " + r);
            i++;
        }
    }
}

class PCFixed {
    public static void main(String args[]) {
        Q q = new Q();
        new Producer(q);
        new Consumer(q);
        System.out.println("Press Control-C to stop.");
    }
}

```

